LIGHTWEIGHT PREFETCH-RESILIENT LLC REPLACEMENT POLICY

Presentation by Sharma Chirag
Roll No. : S20240010223
Guide : Dr . Bheemappa Halavar

Overview

01 Literature Review

02 Problem Statement

03 Objectives

04 Progress Achieved so far

Literature Review

Paper 1: Drishti — MICRO 2025

- Proposes **per-core Global Prediction History Table (GPHT)** for reuse prediction
- Introduces **Dynamic Sampled Cache Selection** — tests multiple policies on sample sets and picks the best
- **Key insight:** Modern LLCs are sliced; replacement policies must account for per-core behavior
- **Not a replacement policy itself** — works as a framework with underlying policies (Mockingjay, Hawkeye)
- **Gap identified:** Drishti does not explicitly distinguish prefetch vs. demand blocks; prefetch-awareness must come from the underlying replacement policy.

Paper 2: PACMan — IEEE Computer Architecture Letters, 2023

- Explicitly addresses **prefetch-demand interaction** at LLC level
- Proposes tracking prefetch accuracy to modulate insertion/promotion policies
- Demonstrates that prefetch-oblivious policies leave significant performance on the table
- **Gap identified:** PACMan's accuracy tracking requires additional counters and sampling logic. There is no study of whether a simpler mechanism (e.g., insertion priority alone) can capture most of the benefit at zero overhead

Summary of Gaps

G a p	Source
No lightweight, zero-overhead prefetch-resilient LLC replacement design exists	PACMan
Drishti's framework does not handle prefetch type needs underlying policy support	Drishti
Prefetch–replacement interaction is understudied in shared multicore LLCs with heterogeneous workloads	Drishti, PACMan

Problem Statement

- Aggressive hardware prefetching can **pollute the Last-Level Cache** by filling it with speculatively fetched blocks that are never accessed. In shared multicore LLCs, this pollution is amplified as multiple cores prefetch simultaneously, causing **eviction of useful demand** data and increasing miss rates.
- Existing solutions either:
- **Ignore the problem** (standard LRU treats prefetch and demand equally)
- **Add significant hardware** (PACMan: accuracy counters and sampling logic; Drishti: GPHT tables + sampled cache selection)
- There is a need for a **lightweight, zero-overhead mechanism** that can reduce prefetch pollution in shared LLCs without requiring additional hardware structures, training periods, or workload-specific tuning.

Objective

1. Design a Zero-Overhead Prefetch-Aware LLC Replacement Policy

- Modify LRU insertion to differentiate **prefetch blocks** (low priority) from **demand blocks** (normal MRU priority)
- Promote prefetched blocks to MRU **only on hit** (proving they are useful)
- Achieve this using **only the existing `access_type` flag —** no new hardware tables or counters

2. Evaluate Across Diverse Workload Types and Multicore Configurations

- Test on **compute-bound** (perlbench), **mixed** (gcc), and **memory-bound** (mcf) SPEC CPU traces
- Evaluate in both **single-core** and **4-core shared LLC** configurations
- Measure **IPC improvement**, **LLC MPKI**, and a new metric: **prefetch usefulness** (promoted prefetches / total prefetch fills)

3: Characterize When Prefetch-Aware Insertion Is Effective vs. When It Is Not

- Quantify prefetch usefulness across workload types to understand when the policy helps
- Demonstrate **self-adaptive behavior —** the policy automatically adjusts based on whether prefetches receive cache hits
- Provide guidance on when lightweight prefetch-awareness suffices vs. when more complex policies (Drishti, PACMan) are needed

Progress Achieved so far

SETUP & IMPLEMENTATION

- ChampSim environment configured (WSL2) with baseline and prefetch-aware binaries (1-core & 4-core)
- Custom replacement module: lru_prefetch_aware (lru_prefetch_aware.h, lru_prefetch_aware.cc)
- Three key functions: find_victim(), replacement_cache_fill(), update_replacement_state()

SINGLE-CORE RESULTS:

Trace	Type	Baseline IPC	Prefetch-Aware IPC	Prefetch Usefulness
perlbench	Compute-bound	2.158	2.159 (+0.05%)	0%
gcc	Mixed	1.967	1.976 (+0.46%)	35.0%
mcf	Memory-bound	0.0743	0.0743 (+0.07%)	0.2%

Result

1-CORE MCF LONG RUN (10M instructions):

Metric	Baseline LRU	Prefetch-Aware LRU
IPC	0.0813	0.08127
LLC Misses	834,521	834,918
Prefetch Usefulness	N/A	0.19% (1 of 538 useful)

4-CORE HETEROGENEOUS (gcc + mcf + perlbench + gcc), Shared 8MB LLC:

Core	Trace	Baseline IPC	Prefetch-Aware IPC
CPU 0	gcc	1.970	1.971
CPU 1	mcf	0.0847	0.0856
CPU 2	perlbench	2.027	2.17
CPU 3	gcc	1.985	1.996

4-Core LLC Statistics:

Prefetch fills: 6,475

Promoted (useful): 2,088

Usefulness: 32.2%

mcf load misses reduced by 226 (318,629 → 318,403)

KEY FINDING: SELF-ADAPTIVE WITHOUT ANY PREDICTOR

gcc (stride-friendly)

Prefetch arrives → Insert LOW

Gets HIT (35%) → Promote to MRU

Result: IPC +0.46%

mcf (pointer-chasing)

Prefetch arrives → Insert LOW

No hits (0.2%) → Stays near LRU

Result: Demand data protected

Remaining work

1. Compare with DRRIP, SHiP, SRRIP
2. More aggressive prefetchers (SPP/AMPM at LLC level)
3. Larger instruction windows (50M+ runs)
4. Adaptive insertion position (lightweight dynamic counter)
5. Drishti integration (per-core reuse prediction + prefetch-awareness)
6. Final report and analysis

Thank You